

SSH URL: `git@github.com:KCalvey/jhu_software_concepts.git`

1. Project Overview

Module 5 adds software assurance and security controls to the GradCafe analytics application developed in the previous modules. The primary focus of this module was improving SQL security, limiting database privileges, analyzing project dependencies, performing automated security scans, and incorporating these checks into continuous integration.

The project uses Pylint for code-quality validation, Pytest for automated testing, pydeps and Graphviz for dependency analysis, Snyk for dependency vulnerability scanning, Snyk Code for static application security testing, and GitHub Actions for continuous integration.

2. Installation and Packaging

The project dependencies can be installed using either `pip` or `uv`.

Using `pip`:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Using `uv`:

```
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
```

The project also includes a `setup.py` file so that Module 5 can be installed as a Python package.

Packaging is important because it makes the project easier to install and reproduce in different environments. It provides a consistent project structure and reduces reliance on machine-specific configurations. Maintaining dependencies in `requirements.txt` also allows development, testing, CI, and security-scanning environments to use the same required packages.

3. Environment-Based Database Configuration

Database credentials are loaded from environment variables rather than being hard-coded into the application source code. The application uses the following variables:

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASSWORD
```

A `.env.example` file is included in the project to demonstrate the required configuration without exposing actual credentials. The real `.env` file is excluded from version control through `.gitignore`.

For the least-privileged configuration, the example file uses the `module5_app` database user. This approach reduces the risk of accidentally exposing database credentials in GitHub or other source-control systems.

4. SQL Injection Defenses

The SQL code was designed to prevent SQL injection by avoiding SQL statements constructed from user input through f-strings, string concatenation, or `.format()`.

Instead, user-supplied values are passed as parameters to the database driver. Psycopg SQL composition tools, including `sql.SQL` and `sql.Identifier`, are used when SQL structure must be constructed dynamically.

SQL statement construction is also kept separate from statement execution. This ensures that user-provided values are treated as data instead of executable SQL.

These defenses reduce the possibility that malicious input could alter the intended structure or behavior of a database query.

5. Query LIMIT Enforcement

Database queries include `LIMIT` clauses to prevent unexpectedly large result sets.

Dynamic limit values are validated through `safe_limit()`. The application restricts requested limits to an allowed range between **1 and 100**, with `MAX_LIMIT` set to **100**.

This provides an additional safeguard against excessive queries and helps protect application and database resources.

6. Least-Privilege Database Access

A dedicated PostgreSQL application role named `module5_app` is defined in `least_privilege.sql`.

The role is configured as:

```
CREATE ROLE module5_app
WITH LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE
NOINHERIT;
```

The role receives only the permissions required by the application:

```
GRANT CONNECT ON DATABASE module3_db TO module5_app;
GRANT USAGE ON SCHEMA public TO module5_app;

GRANT SELECT, INSERT, UPDATE
ON ALL TABLES IN SCHEMA public
TO module5_app;
```

```
REVOKE CREATE ON SCHEMA public FROM module5_app;
```

This follows the principle of least privilege because the application account does not receive superuser, database-creation, or role-creation privileges. It is given only the access necessary to perform the application's normal database operations.

Limiting database privileges reduces the potential impact of a programming error or compromised application account.

7. Dependency Analysis

The project dependency graph was generated using **pydeps** and **Graphviz** with:

```
pydeps src --noshow -o dependency.svg
```

The resulting graph is stored as `dependency.svg`.

The dependency graph provides a visual representation of relationships between Python modules in the project. It makes it easier to identify coupling between components and understand which modules may be affected when another component changes.

Dependency analysis can also reveal unexpected relationships that may increase maintenance or security risks. Because the graph is generated directly from the source code, it can be regenerated as the application changes. The GitHub Actions workflow also generates and validates `dependency.svg` as part of the CI process.

[Insert `dependency.svg` here if you want the graph displayed in the report.]

8. Pylint and Automated Testing

Pylint was executed using the required threshold:

```
pylint src/*.py --fail-under=10
```

The final result was:

Pylint score: 10.00/10

Pytest was also executed against the project test suite.

The final test results were:

82 tests passed

The coverage report showed:

100% test coverage

All **521 statements** measured by the coverage report were covered, with **0 missed statements**.

These automated checks help ensure that the project maintains consistent code quality and that security and application functionality continue to work as expected.

9. Snyk Dependency Security Analysis

Dependency security scanning was performed using:

```
snyk test
```

The scan examined **52 dependencies** and identified **8 issues across 20 vulnerable dependency paths**.

Examples of dependencies identified by the scan included:

- `requests 2.32.5`, with an upgrade to `requests 2.33.0` identified by Snyk.
- `soupsieve 2.8.4`, with an upgrade to `soupsieve 2.9` identified.
- `urllib3 2.6.3`, which contained several medium- and high-severity findings, with an upgrade to `urllib3 2.8.0` identified.

The reported findings included issues involving temporary-file handling, regular-expression denial of service, resource allocation, decompression, certificate validation, and sensitive-data handling.

The complete scan output is included in `snyk_results.txt`, and screenshot evidence is included as `snyk-analysis.png`.

The appropriate remediation is to update affected direct or transitive dependencies to compatible patched versions when possible and then rerun both the automated test suite and Snyk to verify that the vulnerabilities have been addressed without breaking application functionality.

```
(.venv) karicalvey@Mac module_5 % snyk test
Testing /Users/karicalvey/jhu_software_concepts/module_5...

Tested 52 dependencies for known issues, found 8 issues, 20 vulnerable paths.

Issues to fix by upgrading dependencies:

Pin requests@2.32.5 to requests@2.33.0 to fix
x Insecure Temporary File [Medium Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-REQUESTS-15763443] in requests@2.32.5
  introduced by sphinx@7.4.7 > requests@2.32.5 and 2 other path(s)

Pin soupsieve@2.8.4 to soupsieve@2.9 to fix
x Regular Expression Denial of Service (ReDoS) (new) [Medium Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-SOUPSIEVE-19963953] in soupsieve@2.8.4
  introduced by BeautifulSoup4@4.15.0 > soupsieve@2.8.4
x Regular Expression Denial of Service (ReDoS) (new) [Medium Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-SOUPSIEVE-19963967] in soupsieve@2.8.4
  introduced by BeautifulSoup4@4.15.0 > soupsieve@2.8.4

Pin urllib3@2.6.3 to urllib3@2.8.0 to fix
x Infinite Loop (new) [Medium Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-URLLIB3-20302845] in urllib3@2.6.3
  introduced by sphinx@7.4.7 > requests@2.32.5 > urllib3@2.6.3 and 2 other path(s)
x Insertion of Sensitive Information Into Sent Data [High Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-URLLIB3-16642024] in urllib3@2.6.3
  introduced by sphinx@7.4.7 > requests@2.32.5 > urllib3@2.6.3 and 2 other path(s)
x Decompression Bomb [High Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-URLLIB3-16642059] in urllib3@2.6.3
  introduced by sphinx@7.4.7 > requests@2.32.5 > urllib3@2.6.3 and 2 other path(s)
x Improper Certificate Validation (new) [High Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-URLLIB3-20302844] in urllib3@2.6.3
  introduced by sphinx@7.4.7 > requests@2.32.5 > urllib3@2.6.3 and 2 other path(s)
x Allocation of Resources Without Limits or Throttling (new) [High Severity] [https://security.snyk.io/vuln/SNYK-PYTHON-URLLIB3-20302846] in urllib3@2.6.3
  introduced by sphinx@7.4.7 > requests@2.32.5 > urllib3@2.6.3 and 2 other path(s)

Organization:      kcalvey
Package manager:   pip
Target file:       requirements.txt
Project name:      module_5
Open source:       no
Project path:      /Users/karicalvey/jhu_software_concepts/module_5
Licenses:          enabled

Tip: Detected multiple supported manifests (1), use --all-projects to scan all of them at once.
```

10. Snyc Code Static Application Security Testing – Extra Credit

Test Summary

```
Organization:      kcalvey
Test type:         Static code analysis
Project path:      /Users/karicalvey/jhu_software_concepts/module_5

Total issues:      4
Ignored issues:    0 [ 0 HIGH  0 MEDIUM  0 LOW ]
Open issues:       4 [ 0 HIGH  0 MEDIUM  4 LOW ]
```

Snyk Code was also used to perform static application security testing:

```
snyk code test
```

The SAST scan identified **4 LOW-severity findings**, with **0 medium** and **0 high-severity findings**.

The findings included path-traversal warnings involving command-line supplied paths flowing into file operations. Findings were identified in `llm_hosting/app.py` and `src/capture_pages.py`.

The complete SAST output is saved in `snyk_code_results.txt`.

A potential remediation for these findings would be to validate and restrict externally supplied file paths before performing file operations. Paths could be resolved against an approved base directory and rejected if they attempt to access locations outside that directory.

11. GitHub Actions Continuous Integration

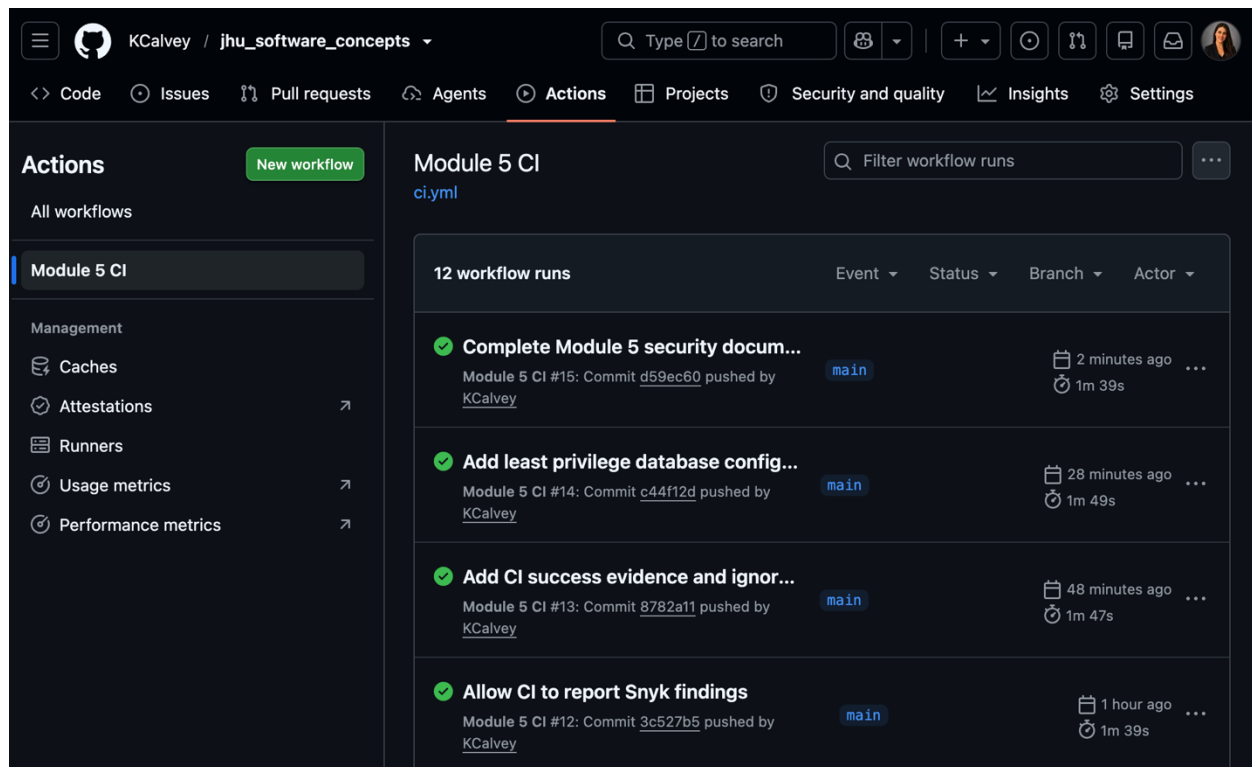
The project includes a GitHub Actions workflow located at:

```
.github/workflows/ci.yml
```

The CI workflow automatically performs the major quality and security checks for Module 5. It:

- Installs the project dependencies and Graphviz.
- Initializes the PostgreSQL test database.
- Runs Pylint with the required `--fail-under=10` threshold.
- Executes the Pytest test suite.
- Generates and validates `dependency.svg`.
- Performs the Snyk dependency security scan.

The GitHub Actions workflow has completed successfully on the `main` branch. Automating these checks makes the security and quality process repeatable and helps identify problems whenever changes are pushed to the repository.



12. Conclusion

Module 5 adds multiple layers of software assurance to the GradCafe analytics project. Parameterized SQL and query limits help protect database operations, while environment variables and a least-privileged PostgreSQL role reduce credential and authorization risks.

Pylint and Pytest provide automated code-quality and functional validation, pydeps documents the project's dependency relationships, and Snyk identifies security concerns in both dependencies and source code. GitHub Actions brings these controls together by automatically repeating the required checks whenever changes are pushed to the repository.

Together, these controls create a more secure, reproducible, and maintainable application.

Supporting Module 5 files include: `dependency.svg`, `snyk-analysis.png`, `snyk_results.txt`, `snyk_code_results.txt`, `least_privilege.sql`, `.env.example`, `requirements.txt`, `setup.py`, `pytest.ini`, and the GitHub Actions `ci.yml` workflow.